一、数据解析与心率时间序列构建

通过读取原始数据集，容易发现它包含了 15869 行和 7 列，其中包括了来自多种设备的记录。首先按照 Key 字段筛选 heart_rate 记录，再从 Value 字段中提取 time 和 bpm，分别作为记录时间和每分钟心跳次数。整理成时间序列后，就可以继续检查数据质量，观察心率随时间的变化，并比较重采样与滑动平均的结果。

考虑使用正则表达式从 Value 字段提取 time 和 bpm 的值。提取出的 time 是以秒为单位的 Unix 时间戳，先按 UTC 转换为时间对象，再转为 Asia/Shanghai 时区，得到北京时间。最后去掉时区标记，方便后续按日期筛选和计算，此时保留的就是转换后的北京时间。对应的关键代码如下代码 1 所示。

代码 1　正则提取心率并转换为北京时间

```
values = raw.loc[raw["Key"] == "heart_rate", "Value"]
df = pd.DataFrame()
# 分别提取时间戳与心率，转换为整数
df["time"] = values.str.extract(r'"time"\s*:\s*(\d+)')[0]
df["bpm"] = values.str.extract(r'"bpm"\s*:\s*(\d+)')[0]
df = df.dropna().astype("int64")
# 先按 UTC 解释，再转换为北京时间 0
df["time"] = pd.to_datetime(df["time"], unit="s", utc=True)
df["time"] =
df["time"].dt.tz_convert("Asia/Shanghai").dt.tz_localize(None)
```

筛选后共得到 10355 条心率记录，time 和 bpm 全部提取成功，两个字段均无缺失。随后按照 time 升序排列，并检查重复时间，处理后仍保留 10355 条，说明这组心率记录没有重复时间点。整理结果保存为 heart_rate_time_series.csv，仅保留 time 和 bpm 两列。后续读取这个文件，将 time 设为索引，就能直接按日期取出记录，也能按实际经过的时间进行重采样和滑动计算。

二、数据质量体检

完成时间转换后，先查看记录覆盖了哪些时段，再检查相邻记录的间隔和心率数值，下表 1 列出了初步的检查情况。其中，异常值是通过 1.5IQR 的范围来界定的，即不在 31.5bpm 到 131.5bpm 之间的心率值会被视为异常值。

表 1　初步体检结果

检查项	检查结果
记录规模	10355 条记录
采样间隔	1 分钟 (97.02%)
缺失段数	10 段
异常值	33 条 (0.32%)

对于采样间隔，统计得出一共有约一万个间隔，其中 1 分钟间隔的占比是最多的，其次是 2 分钟间隔（2.46%），然后是 3 分钟间隔（0.07%）。实际发现还有很多奇怪的采样间隔，但它们的占比都很小，这部分小占比的已在下表 2 列出。

表 2　小占比的采样间隔

采样间隔 (分钟)	占比	采样间隔 (分钟)	占比
4	0.02%	9	0.02%
10	0.25%	6	0.01%
8	0.01%	37	0.01%
39	0.01%	44	0.01%
50	0.01%	54	0.01%
78	0.01%	95	0.01%
98	0.01%	107	0.01%
144	0.01%	221	0.01%
319	0.01%	506	0.01%
888	0.01%	7847	0.01%

将上面采样间隔大于 60 分钟的视为缺失段，则一共存在 10 个超过 1 小时的缺口，具体的缺口如下表 3 所示。

表 3　所有缺口的时段和持续时长

起始时间	终止时间	持续时长（分钟）
05/09 22:44	05/09 00:02	78
05/10 19:25	05/11 10:13	888
05/13 22:15	05/14 00:39	144
05/14 22:03	05/14 23:41	98
05/15 10:32	05/15 14:13	221
05/16 22:18	05/16 03:37	319
05/16 12:16	05/16 13:51	95
05/16 17:52	05/17 02:18	506
05/17 12:01	05/17 13:48	107
05/18 13:14	05/24 00:01	7847

关于代码实现细节，采样间隔其实是通过相邻时间相减得到的，然后再统计各间隔的次数。完成后，再将超过一小时的部分单独列出。对应的核心代码如下代码 2 所示。

代码 2　检查采样间隔和长时间缺口

```
# 将相邻观测的时间差换算为分钟
interval =
heart.index.to_series().diff().dt.total_seconds().div(60)
frequency = interval.value_counts().sort_index()
# 超过一小时的间隔单独检查
gaps = interval[interval > 60]
```

对于描述性统计，心率数据集中，均值是 82.70bpm，中位数为 84bpm，样本标准差为 16.28bpm，最小值为 38bpm，最大值为 168bpm，对应的 Q1、Q2 和 Q3 四分位数分别为 69bpm，84bpm 和 94bpm。

综上所述，心率记录中有几处较长的中断，最长超过五天，另外还有 33 条偏高的心率记录。记录中断会使重采样后的部分时间段没有数据，本次将这些时段留空，只根据实际记录计算统计量。不过，各时间段的记录数量不同，记录较少时，算出的平均心率更容易受到个别数值的影响。那 33 条高值也会拉高平均心率和标准差，考虑到它们可能是实际活动时的心率，本次暂时保留，后续结合中位数和原始心率曲线一起分析，而不是将这些偏高的心率直接剔除。

三、心率时间序列的多维可视化

前面的检查给出了数据的基本情况，接下来就可以可视化心率变化从而理解数据集。我们首先考虑用折线图查看心率随时间的变化，然后用直方图了解心率主要集中在哪个范围，以及箱线图观察数值的波动和偏高或偏低的记录，最后用饼图展示不同心率区间的记录各占多少。这样既能看出哪些时段的心率变化较大，也能了解大多数记录处于什么水平。

绘制折线图时，先补齐一分钟时间索引，缺少记录的位置保留为空值，曲线就会在缺口处断开。这样既保留了观测点之间的先后关系，也能清楚显示哪些时段没有采集到心率，尤其适合展示本次数据中的长时间中断。关键处理如下代码 3 所示。

代码 3　绘制心率随时间的变化图

```
# 只补齐时间索引，心率缺测位置保持为空
timeline = heart.asfreq("1min")
plt.plot(timeline.index, timeline, lw=0.7)
```

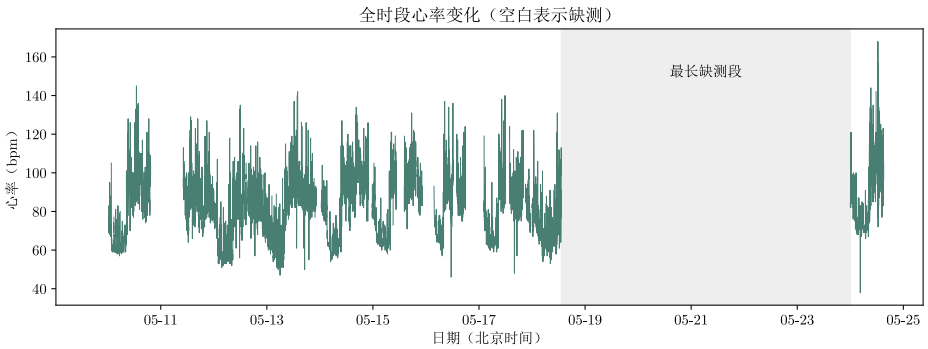


图 1　全时段心率变化

从图 1 来看，心率大多在一定范围内波动，有些时段会短暂升高。比较明显的是 5 月 18 日至 5 月 24 日之间空了一大段，正好对应前面发现的最长缺口，这几天的心率变化是未知的。对于局限性，图 1 将所有

记录放在一张图里，虽然可以从时间变化看出整体的趋势变化，但心率数据是按 1 分钟采样的，而图 1 包含了好几天的数据，导致数据在图中呈现地很密集。例如，如果要问一次高峰是如何形成和消失的，就很难直接从图 1 进行理解，因为它太密集，难以观察细节。

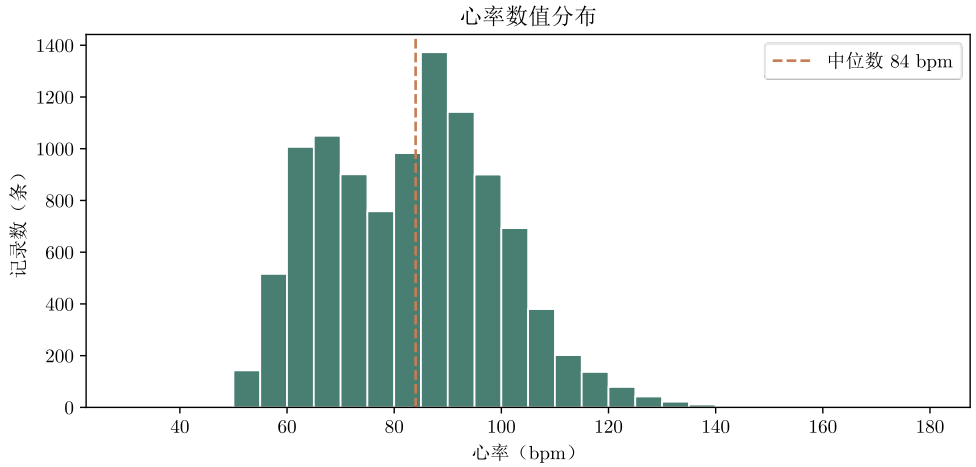


图 2　　心率数值分布

图 2 中，大多数心率记录集中在 60bpm 到 100bpm 之间，中位数为 84bpm。超过 130bpm 的记录比较少，最高值虽然达到了 168bpm，但这样的高值在整组数据中只占很小一部分。

平均心率约为 82.70bpm，与中位数 84bpm 比较接近，两者都在 80 多 bpm。因此，尽管个别记录达到了较高水平，这组数据的平均心率仍然比较接近记录最集中的区域。

对于局限性，图 2 虽然把不同日期和时段的记录合在一起展示了，可以很直观的看出哪些心率数值是常见的，哪些不常见，但是这些数据对应的时间却丢失了，例如，同样是高心率记录，可能集中在一次活动中，也可能分散在几天里，这些情况在直方图中很难区分。

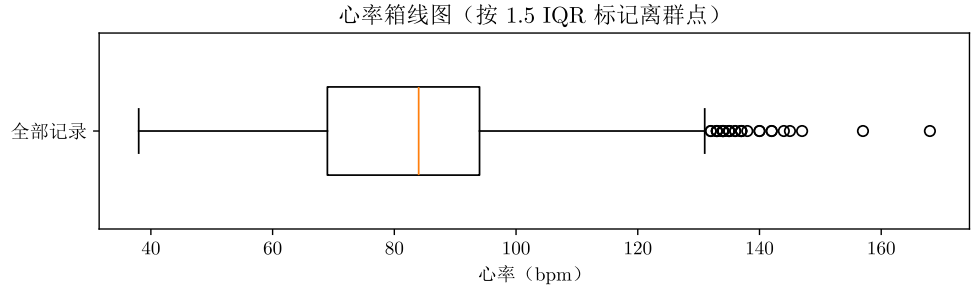


图 3　　心率箱线图

图 3 中，一半的心率记录落在 69bpm 到 94bpm 之间，中位数为 84bpm，和前面直方图反映的情况基本一致。右侧单独标出的 33 条记录

都超过了 131.5bpm，比大多数记录明显偏高。对于局限性，箱线图能简洁地概括心率的整体分布，但也省略了很多细节。例如，这 33 条高值是集中在某一天，还是分散在不同日期，图中就看不出来。除此外，69bpm 到 94bpm 之间哪些数值出现得更多，也无法直接判断。因此，分析高值出现的时间需要结合折线图，而如果要分析具体的分布情况，则需要结合直方图。

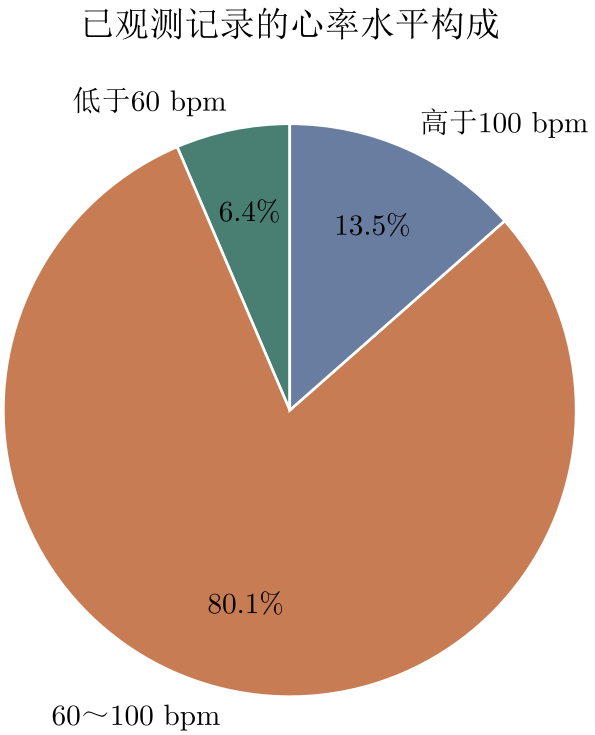


图 4　已观测记录的心率水平构成

从下图 4，可以看出有 80.1%的数据集中在 60bpm 到 100bpm 之间，随后是高于 100bpm 的，占 13.5%，最后是低于 60bpm，占 6.4%。因此，大多数情况下，心率都在 60bpm 到 100bpm 之间，这与我们之前几个图的观察结果基本一致。这张饼图的局限是，这张图按记录条数统计，不能反映各心率区间实际持续了多久，并且由于采样间隔不一致且存在长时间缺失，图中的占比和时间占比是表达的含义是不同的，不能简单的把它也认为是时间上的占比。

四、单日心率折线图

现在分析 2024 年 5 月 10 日单日的心率变化。按这个日期筛选后得到了 1071 条记录，时间上覆盖当天 00:02 到 19:25。相关核心代码如下代码 4 所示。

代码 4　选取单日记录并确定纵轴范围

```
# 按北京时间筛选五月十日的观测
day = heart.loc["2024-05-10"]
series = day.asfreq("1min")
# 在当天最小值和最大值两侧各留出 10 bpm
ax.set_ylim(day.min() - 10, day.max() + 10)
```

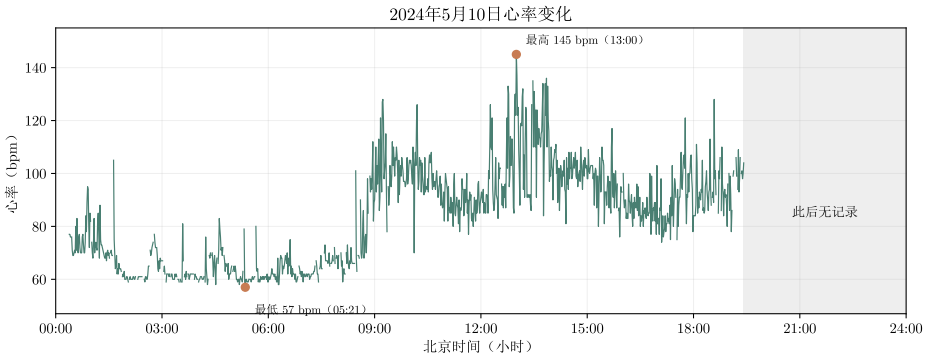


图 5　　当日心率变化图

当天已记录心率的均值为 83.784bpm，样本标准差为 18.256bpm。最低值为 57bpm，出现在当日 05:21，而最高值为 145bpm，出现在当日 13:00。由此，纵轴设置为 47bpm 到 155bpm 之间，以供展示全貌。

从图 5 可以看出，凌晨至清晨的心率相对较低，曲线也比较平稳，可以暂时看作睡眠阶段。随后八点过十分，心率有所升高，起伏逐渐增多，可能对应醒来后开始活动的过渡过程。到了上午九点左右，心率波动更加明显，期间出现短时高峰，可能与走动或其他活动有关。

五、重采样与聚合函数对比

单日图中的心率变化比较密集，这里按十分钟划分时间段，把每段内的记录汇总后，再看看整体变化。汇总时分别计算均值、中位数、最大值和样本标准差，观察不同方法得到的曲线有什么差别。接着检查重采样后是否出现空值，并比较保留空值、线性插值和前向填充三种处理方式，关键代码如下代码 5 所示。

代码 5　十分钟聚合及缺失窗口处理

```
# 各时间段分别计算四种统计量
aggregation = day.resample("10min").agg(["mean", "median",
"max", "std"])
mean = aggregation["mean"]
# 只在已有边界内部插值，另一种方法延用前值
linear = mean.interpolate(method="linear", limit_area="inside")
forward = mean.ffill()
```

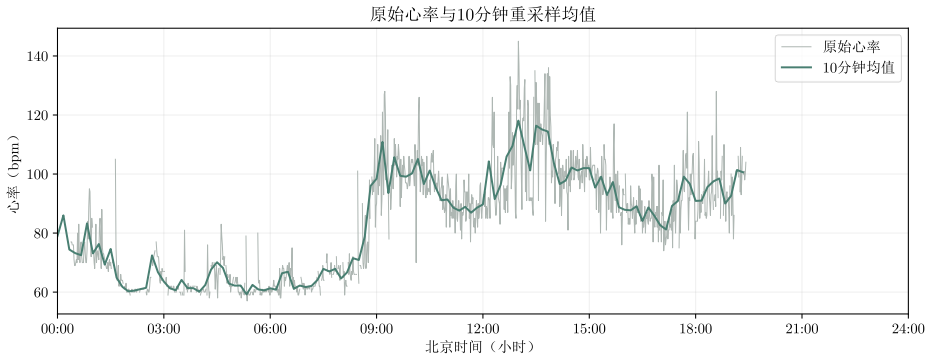


图 6　　原始心率与十分钟重采样均值

重采样后共得到 117 个结果，从 00:00 开始，每十分钟划分一段，最后一段从 19:20 开始。例如，00:02 的记录会与 00:00 至 00:10 之间的其他记录一起计算均值。图 6 中，十分钟均值保留了清晨心率较低、白天逐渐升高的变化，但原始曲线中的尖峰变低了。最高的十分钟平均心率为 118.1bpm，而原始记录的最高值为 145bpm。这是因为短时高值与同一时段内的其他记录一起参与了平均，因此，重采样后的曲线更适合观察各时段的整体心率水平。

对 117 个窗口均值再次求平均，得到 83.09bpm，略低于原始记录均值 83.78bpm。原因在于两种计算的权重不同，即，原始均值让每条记录具有相同权重，窗口均值再平均则让每个窗口具有相同权重。

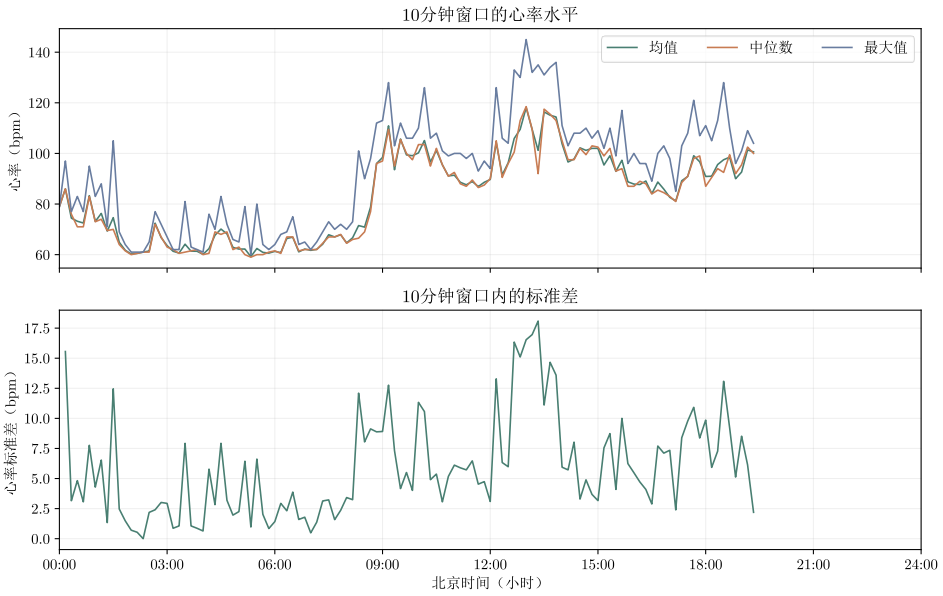


图 7　　十分钟窗口的聚合函数比较

图 7 中，均值和中位数两条曲线的变化趋势是比较接近的，并且心率升高和回落的时段基本一致。把各个十分钟区间的均值和中位数分别再求平均，得到 83.09bpm 和 82.66bpm，从图上来看其实差不太多。然

而，最大值曲线很显然是区别更大的，各段最大值的平均数为 92.77bpm，其中一些短时高峰也更加明显的。均值和中位数能帮助我们了解这十分钟内心率大致处于什么水平，但有些高值经过平均后就不明显了。

前三种统计量反映心率水平，标准差则反映同一窗口内的波动，所以图 7 将标准差单独放在下方子图。其有效窗口数为 116，窗口内标准差平均为 5.90bpm，最大为 18.09bpm。在标准差较高的时段，窗口内数值更分散。

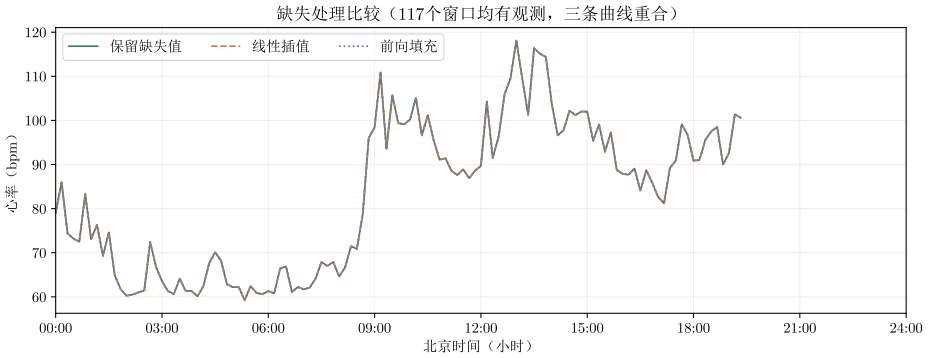


图 8　　十分钟窗口的缺失处理比较

图 8 中的三条曲线重合。检查后发现，实际记录范围内的 117 个十分钟窗口都至少有一条观测，窗口均值没有空值，所以插值和前向填充都保留了原结果，三种方式的均值、标准差和极值也完全相同。前面提到的分钟级缺测，在十分钟聚合后可能与同窗口内的其他记录合并，因此这次没有形成空窗口。

这次只对实际有记录的时间范围进行重采样，最后一段从 19:20 开始，19:25 之后没有记录，也就不再往后补值。如果其他日期出现较短的空缺，可以考虑用前后记录估计中间的数值，也就是线性插值。当然，也可以选择直接沿用上一次的数值，即前向填充。具体采用哪种方法，还要看缺失前后的心率变化。这组数据最长的一段空缺超过五天，期间可能有睡眠、走动等多种情况，仅靠缺口两端的记录很难推知中间的心率，因此这里保留空缺。

六、滑动窗口与相位滞后

十分钟重采样把一天分成不重叠的时间段，接下来用滑动平均进一步观察连续变化。这里采用三十分钟时间窗口，分别计算后向平均和居中平均。后向窗口主要使用当前及此前的记录，居中窗口使用当前时刻前后约十五分钟的记录。相应的关键代码如下代码 6 所示。

代码 6　后向与居中的三十分钟滑动平均

```
# 至少有一条有效观测时就计算窗口均值
rolling = pd.DataFrame(
    {
```

```
        "后向平均": day.rolling("30min", min_periods=1).mean(),
        "中心平均": day.rolling("30min", center=True,
min_periods=1).mean(),
    }
)
```

其中，min_periods=1 表示窗口内有一条有效观测就计算均值，这样序列起点也能得到结果。记录较少的窗口参与平均的数值更少，平滑程度会与记录充分的窗口有所区别。计算后仍使用原始时间位置，缺少记录的分钟位置在图中保持空白，方便与原始曲线对照，结果如图 9 所示。

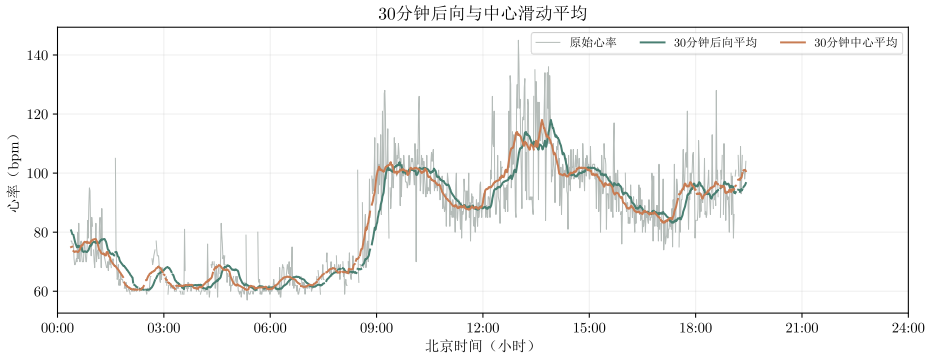


图 9　　三十分钟滑动平均

平滑后，曲线中的短时起伏减少了。原始心率的样本标准差为 18.26bpm，后向平均和居中平均分别降到了 16.48bpm 和 16.56bpm。原始记录在 13:00 达到最高值 145bpm，而两条平滑曲线的最高值都降到了 118.03bpm，出现时间分别为 13:55 和 13:40，比原始最高点晚了 55 分钟和 40 分钟。

后向平均用当前及之前一段时间的记录计算均值。心率刚开始升高时，前面较低的数值还在参与计算，因此平均值上升得较慢，曲线就会有些滞后。不过，这次最高点推迟还有另一个原因，因为 13:00 的尖峰持续时间较短，经过平均后被压低了，后面持续较久的一段高心率反而成了平滑曲线的最高点。所以，55 分钟和 40 分钟只是这几条曲线最高点之间的时间差，其他高峰经过平滑后，不一定也会推迟这么久。

居中平均把当前时刻前后的记录放在一起计算，通常能减轻后向平均的滞后，更适合在数据采集完成后查看变化。例如，13:40 的居中均值会用到之后约 15 分钟的记录。实时分析时，后面的数据还没有产生，可以使用后向平均，也可以用 `day.ewm(span=x, adjust=False).mean()` 计算指数加权平均，让较近的记录占更大权重。

七、数据解读与综合结论

本次实验从原始数据中提取了 10355 条心率记录，将时间统一转换为北京时间，再进行检查，绘图和后续处理。大部分记录间隔为一分钟，

但中间也有一些较长的中断，其中超过一小时的有 10 段，最长的五天以上的都有。各天记录的完整程度也不同，有些日期只有部分时段的数据。

从已有记录来看，一半的心率落在 69bpm 到 94bpm 之间，60 的到 100bpm 的记录占 80%左右，较高的数值只占少数。按 IQR 规则筛出的 33 条高值本次都予以保留，再结合心率曲线观察。5 月 10 日的心率在清晨较低，白天有所升高，期间出现了几次明显的波动，可能与醒来后活动增加有关。

几种处理方法得到的曲线都各有自己的用途，例如，原始曲线保留了短时起伏，能看清心率最高到了多少。按十分钟求均值后，曲线更平缓，各时段的大致水平也更容易比较，但一些短时高峰会被压低，这时可以结合最大值曲线查看。标准差则能帮助找出心率波动较大的时段。三十分钟滑动平均进一步减少了起伏，不过，平滑后最高点出现的时间也发生了变化。因此，如果想分析某一次心率升高，可以先从平滑曲线中找到大致时段，再回到原始记录中查看具体时间和幅度。

对于缺失值的处理，5 月 10 日虽然有些分钟没有记录，但每个十分钟区间内都有数据，重采样后没有出现空值，所以保留空值、线性插值和前向填充得到的结果完全相同。对于晚间未记录的部分，以及跨越数小时甚至数天的缺口，本次都保留为空。这些时段内可能发生过多次休息和活动，仅靠缺口前后的心率很难补出可靠的数值。

八、AI 工具使用声明

本人未与他人讨论本实验，但使用了 OpenAI Codex 的 GPT 6.1 Sol 模型辅助代码编写与修改，也包括图表绘制、结果分析和报告文字的草拟。在这基础上，再经过大量人工修改、复核和排版后撰写出本文。

九、附录

analysis.py　运行入口

```python
from step_one_prepare import run_step_one
from step_two_quality import run_step_two
from step_three_visualization import run_step_three
from step_four_daily import run_step_four
from step_five_resampling import run_step_five
from step_six_rolling import run_step_six

def run_experiment() -> None:
    """按作业顺序完成解析、质量检查和可视化。"""
    heart = run_step_one()
    run_step_two(heart)
    run_step_three(heart)
    day = run_step_four(heart)
    run_step_five(day)
    run_step_six(day)
```

```
    print("\n 已完成实验，SVG 图片保存在 Week 7/figures 中。")

if __name__ == "__main__":
    run_experiment()
```

common.py　统一数据路径与读取函数

```
from pathlib import Path
import pandas as pd

class Config:
    ROOT: Path = Path(__file__).resolve().parent.parent
    FIGURES: Path = ROOT / "figures"
    DAY: str = "2024-05-10"

def load_heart_rate() -> pd.Series:
    """读取已转换的心率时间序列。"""
    df = pd.read_csv(Config.ROOT / "heart_rate_time_series.csv",
parse_dates=["time"])
    return df.set_index("time")["bpm"]
```

plot.py　绘制并保存实验图表

```
from pathlib import Path
from typing import Any

import matplotlib.dates as mdates
import matplotlib.pyplot as plt
import pandas as pd

class PlotConfig:
    STYLE: dict[str, Any] = {
        "font.family": ["Latin Modern Math", "SimSun"],
        "mathtext.fontset": "cm",
        "axes.formatter.use_mathtext": True,
        "axes.unicode_minus": False,
        "svg.fonttype": "path",
        "font.size": 11,
    }
    COLORS: list[str] = ["#497f72", "#c87c53", "#697da0"]

plt.rcParams.update(PlotConfig.STYLE)

def save_figure(output_path: Path) -> None:
    """保存当前图片"""
    output_path.parent.mkdir(exist_ok=True)
    plt.tight_layout()
    plt.savefig(output_path, format="svg", bbox_inches="tight")
    plt.close()

def set_day_axis(ax: plt.Axes, day: pd.Series) -> None:
    """统一单日图的小时刻度"""
    start = day.index[0].normalize()
    ax.set_xlim(start, start + pd.Timedelta(days=1))
    ax.set_xticks(pd.date_range(start, periods=9, freq="3h"))
    ax.set_xticklabels([f"{hour:02d}:00" for hour in range(0,
25, 3)])
```

```
    ax.set_xlabel("北京时间（小时）")
    ax.set_ylabel("心率（bpm）")
    ax.grid(alpha=0.2)

def plot_overview(heart: pd.Series, output_dir: Path) -> None:
    """用折线图、直方图和箱线图观察整体情况"""
    plt.figure(figsize=(10, 3.9))
    # 补出一分钟时间索引，缺测处保持空白，避免跨缺口连线。
    timeline = heart.asfreq("1min")
    plt.plot(timeline.index, timeline,
color=PlotConfig.COLORS[0], lw=0.7)
    gaps = heart.index.to_series().diff()
    gap_end = gaps.idxmax()
    gap_start = gap_end - gaps.max()
    plt.axvspan(gap_start, gap_end, color="#eeeeee")
    plt.text(gap_start + gaps.max() / 2, 150, "最长缺测段",
ha="center")
    plt.gca().xaxis.set_major_formatter(mdates.DateFormatter("%m
-%d"))
    plt.xlabel("日期（北京时间）")
    plt.ylabel("心率（bpm）")
    plt.title("全时段心率变化（空白表示缺测）")
    save_figure(output_dir / "01 全时段心率.svg")

    plt.figure(figsize=(8, 4))
    plt.hist(
        heart, bins=range(30, 181, 5),
color=PlotConfig.COLORS[0], edgecolor="white"
    )
    plt.axvline(
        heart.median(),
        color=PlotConfig.COLORS[1],
        ls="--",
        label=f"中位数 {heart.median():.0f} bpm",
    )
    plt.xlabel("心率（bpm）")
    plt.ylabel("记录数（条）")
    plt.title("心率数值分布")
    plt.legend()
    save_figure(output_dir / "02 心率分布.svg")

    plt.figure(figsize=(8, 2.6))
    plt.boxplot(heart, vert=False, widths=0.45)
    plt.yticks([1], ["全部记录"])
    plt.xlabel("心率（bpm）")
    plt.title("心率箱线图（按 1.5 IQR 标记离群点）")
    save_figure(output_dir / "03 心率箱线图.svg")

def plot_daily(day: pd.Series, output_dir: Path) -> None:
    """观察五月十日的日内变化"""
    fig, ax = plt.subplots(figsize=(10, 4))
    series = day.asfreq("1min")
    ax.plot(series.index, series, color=PlotConfig.COLORS[0],
lw=0.9)
```

```
    for time, label, offset in [(day.idxmin(), "最低", -20),
(day.idxmax(), "最高", 8)]:
        ax.scatter(time, day[time], color=PlotConfig.COLORS[1],
zorder=3)
        ax.annotate(
            f"{label} {day[time]:.0f} bpm（{time:%H:%M}）",
            (time, day[time]),
            xytext=(7, offset),
            textcoords="offset points",
            fontsize=9,
        )
    end = day.index[0].normalize() + pd.Timedelta(days=1)
    ax.axvspan(day.index[-1], end, color="#eeeeee")
    ax.text(
        day.index[-1] + (end - day.index[-1]) / 2,
        day.mean(),
        "此后无记录",
        ha="center",
        fontsize=10,
    )
    ax.set_ylim(day.min() - 10, day.max() + 10)
    set_day_axis(ax, day)
    ax.set_title("2024 年 5 月 10 日心率变化")
    save_figure(output_dir / "04 单日心率.svg")

def plot_resampling(
    day: pd.Series,
    aggregation: pd.DataFrame,
    strategies: pd.DataFrame,
    output_dir: Path,
) -> None:
    """比较十分钟重采样、聚合函数及缺失值处理"""
    fig, ax = plt.subplots(figsize=(10, 4))
    original = day.asfreq("1min")
    ax.plot(original.index, original, color="#aeb7b3", lw=0.7,
label="原始心率")
    ax.plot(
        aggregation.index,
        aggregation["mean"],
        color=PlotConfig.COLORS[0],
        lw=1.5,
        label="10 分钟均值",
    )
    set_day_axis(ax, day)
    ax.legend()
    ax.set_title("原始心率与 10 分钟重采样均值")
    save_figure(output_dir / "05 重采样均值.svg")

    fig, axes = plt.subplots(2, 1, figsize=(10, 6.4),
sharex=True)
    for column, name, color in zip(
        ["mean", "median", "max"], ["均值", "中位数", "最大值"],
PlotConfig.COLORS
    ):
        axes[0].plot(
```

```
            aggregation.index, aggregation[column], label=name,
color=color, lw=1.2
        )
    axes[0].set_title("10 分钟窗口的心率水平")
    axes[0].legend(ncol=3)
    axes[1].plot(
        aggregation.index, aggregation["std"],
color=PlotConfig.COLORS[0], lw=1.2
    )
    axes[1].set_title("10 分钟窗口内的标准差")
    for ax in axes:
        set_day_axis(ax, day)
    axes[0].set_xlabel("")
    axes[1].set_ylabel("心率标准差（bpm）")
    save_figure(output_dir / "06 聚合函数比较.svg")

    fig, ax = plt.subplots(figsize=(10, 4))
    for column, color, style in zip(
        strategies.columns, PlotConfig.COLORS, ["-", "--", ":"]
    ):
        ax.plot(
            strategies.index,
            strategies[column],
            color=color,
            linestyle=style,
            lw=1.3,
            label=column,
        )
    set_day_axis(ax, day)
    ax.set_title("缺失处理比较（117 个窗口均有观测，三条曲线重合）")
    ax.legend(ncol=3)
    save_figure(output_dir / "07 缺失窗口处理.svg")

def plot_rolling(day: pd.Series, rolling: pd.DataFrame,
output_dir: Path) -> None:
    """比较后向和居中的三十分钟滑动平均"""
    fig, ax = plt.subplots(figsize=(10, 4))
    original = day.asfreq("1min")
    ax.plot(original.index, original, color="#b5bcb9", lw=0.7,
label="原始心率")
    for column, color in zip(rolling.columns,
PlotConfig.COLORS):
        line = rolling[column].asfreq("1min")
        ax.plot(line.index, line, color=color, lw=1.5, label=f"30
分钟{column}")
    set_day_axis(ax, day)
    ax.set_title("30 分钟后向与中心滑动平均")
    ax.legend(ncol=3, fontsize=9)
    save_figure(output_dir / "08 滑动平均.svg")

def plot_levels(heart: pd.Series, output_dir: Path) -> None:
    """用心率水平构成补充整体观察，不作为活动强度诊断"""
    counts = pd.cut(
        heart,
        [-float("inf"), 59, 100, float("inf")],
```

```
        labels=["低于 60 bpm", "60～100 bpm", "高于 100 bpm"],
    )
    plt.figure(figsize=(7, 4.3))
    plt.pie(
        counts.value_counts(sort=False),
        labels=counts.cat.categories,
        autopct="%1.1f%%",
        startangle=90,
        colors=PlotConfig.COLORS,
        wedgeprops={"edgecolor": "white"},
        pctdistance=0.7,
    )
    plt.title("已观测记录的心率水平构成")
    save_figure(output_dir / "09 心率水平构成.svg")
```

step_one_prepare.py　提取心率记录并转换时间

```
import pandas as pd
from common import Config

def run_step_one() -> pd.Series:
    """筛选心率记录，用正则表达式提取时间和心率。"""
    raw = pd.read_csv(Config.ROOT / "dataofxiaomi.csv")
    values = raw.loc[raw["Key"] == "heart_rate", "Value"]
    df = pd.DataFrame()
    df["time"] = values.str.extract(r'"time"\s*:\s*(\d+)')[0]
    df["bpm"] = values.str.extract(r'"bpm"\s*:\s*(\d+)')[0]
    df = df.dropna().astype("int64")
    print(f"心率原始记录：{len(values)} 条；解析成功：{len(df)} 条
")

    # 先解释为 UTC，再转换为北京时间，最后去掉时区标记。
    df["time"] = pd.to_datetime(df["time"], unit="s", utc=True)
    df["time"] =
df["time"].dt.tz_convert("Asia/Shanghai").dt.tz_localize(None)
    df = df.sort_values("time").drop_duplicates("time")
    df.to_csv(Config.ROOT / "heart_rate_time_series.csv",
index=False)
    print(
        f"去重后记录：{len(df)} 条；时间范围：{df['time'].min()} 至
{df['time'].max()}"
    )
    return df.set_index("time")["bpm"]

if __name__ == "__main__":
    run_step_one()
```

step_two_quality.py　检查记录质量与异常值

```
import pandas as pd
from common import load_heart_rate

def run_step_two(heart: pd.Series) -> None:
    """检查采样间隔、长缺口、描述统计和异常值，不删除观测。"""
    print("\n 二、数据质量体检")
    print(
```

```
        f"有效记录：{len(heart)} 条；有记录日期：
{heart.index.normalize().nunique()} 天"
    )
    print(f"起止时间：{heart.index.min()} 至 {heart.index.max()}")
    span = heart.index.max() - heart.index.min()
    print(
        f"时间跨度：{span.days} 天 {span.components.hours} 小时
{span.components.minutes} 分钟"
    )
    interval =
heart.index.to_series().diff().dt.total_seconds().div(60)
    frequency = interval.value_counts().sort_index().to_frame("次
数")
    frequency["占比（%）"] = frequency["次数"] / interval.count()
* 100
    frequency.index.name = "采样间隔（分钟）"
    print("采样间隔分布：\n" + frequency.round(3).to_string())

    gaps = pd.DataFrame(
        {
            "开始时间": heart.index.to_series().shift(),
            "结束时间": heart.index,
            "间隔（分钟）": interval,
        }
    )
    gaps = gaps.loc[interval > 60]
    print("超过一小时的相邻记录间隔：\n" +
gaps.to_string(index=False))
    print(f"长缺口共 {len(gaps)} 段；最长 {gaps['间隔（分钟）
'].max():.0f} 分钟")

    description = heart.describe().rename(
        index={
            "count": "数量",
            "mean": "均值",
            "std": "标准差",
            "min": "最小值",
            "25%": "第一四分位数",
            "50%": "中位数",
            "75%": "第三四分位数",
            "max": "最大值",
        }
    )
    print("描述统计：\n" + description.round(3).to_string())
    q1, q3 = heart.quantile([0.25, 0.75])
    low, high = q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1)
    outlier = (heart < low) | (heart > high)
    print(
        f"IQR 界限：{low:.1f}～{high:.1f} bpm；异常值
{outlier.sum()} 条，占 {outlier.mean():.3%}"
    )
    print(f"超出 30～220 bpm 的观测：{((heart < 30) | (heart >
220)).sum()} 条")
```

```python
if __name__ == "__main__":
    run_step_two(load_heart_rate())
```

step_three_visualization.py　绘制全时段可视化

```python
import pandas as pd
from common import Config, load_heart_rate
from plot import plot_overview, plot_levels

def run_step_three(heart: pd.Series) -> None:
    """从趋势、分布、离散程度和构成四个角度观察心率。"""
    Config.FIGURES.mkdir(exist_ok=True)
    plot_overview(heart, Config.FIGURES)
    plot_levels(heart, Config.FIGURES)

if __name__ == "__main__":
    run_step_three(load_heart_rate())
```

step_four_daily.py　分析单日心率变化

```python
import pandas as pd
from common import Config, load_heart_rate
from plot import plot_daily

def run_step_four(heart: pd.Series) -> pd.Series:
    """截取指定日期，输出有效时段和极值时刻。"""
    day = heart.loc[Config.DAY]
    print("\n 四、单日心率")
    print(f"日期：{Config.DAY}；有效记录：{len(day)} 条")
    print(f"覆盖时段：{day.index.min():%H:%M}～
{day.index.max():%H:%M}")
    print(f"均值：{day.mean():.3f} bpm；标准差：{day.std():.3f}
bpm")
    for name, value in [("最小值", day.min()), ("最大值",
day.max())]:
        times = day.index[day ==
value].strftime("%H:%M").tolist()
        print(f"{name}：{value} bpm；出现时刻：{'、
'.join(times)}")
    plot_daily(day, Config.FIGURES)
    return day

if __name__ == "__main__":
    run_step_four(load_heart_rate())
```

step_five_resampling.py　比较重采样与缺失处理

```python
import pandas as pd
from common import Config, load_heart_rate
from plot import plot_resampling

def run_step_five(day: pd.Series) -> None:
    """比较十分钟聚合及缺失值处理，不扩展原始记录的边界。"""
```

```
    aggregation = day.resample("10min").agg(["mean", "median",
"max", "std"])
    mean = aggregation["mean"]
    strategies = pd.DataFrame(
        {
            "保留缺失值": mean,
            "线性插值": mean.interpolate(method="linear",
limit_area="inside"),
            "前向填充": mean.ffill(),
        }
    )
    print("\n 五、十分钟重采样")
    print(
        f"窗口总数：{len(mean)}；空窗：{mean.isna().sum()}；每窗最
少观测：{day.resample('10min').count().min()}"
    )
    summary = aggregation.agg(["count", "mean", "std", "min",
"max"]).T
    summary.columns = ["有效窗口", "均值", "标准差", "最小值", "最
大值"]
    print("聚合结果统计：\n" + summary.round(3).to_string())
    comparison = strategies.agg(["count", "mean", "std", "min",
"max"]).T
    comparison.columns = summary.columns
    print("缺失策略比较：\n" + comparison.round(3).to_string())
    print("上述窗口均有数据，因此三种策略结果一致；19:25 后的未观测时
段不作外推。")
    plot_resampling(day, aggregation, strategies,
Config.FIGURES)

if __name__ == "__main__":
    run_step_five(load_heart_rate().loc[Config.DAY])
```

step_six_rolling.py　计算并比较滑动平均

```
import pandas as pd
from common import Config, load_heart_rate
from plot import plot_rolling

def run_step_six(day: pd.Series) -> None:
    """比较三十分钟后向窗口与中心窗口，量化全局峰值时刻差。"""
    rolling = pd.DataFrame(
        {
            "后向平均": day.rolling("30min",
min_periods=1).mean(),
            "中心平均": day.rolling("30min", center=True,
min_periods=1).mean(),
        }
    )
    print("\n 六、三十分钟滑动平均")
    print(f"原始峰值：{day.max():.3f} bpm，时刻
{day.idxmax():%H:%M}")
    for name in rolling.columns:
        series = rolling[name]
```

```
        delay = (series.idxmax() - day.idxmax()).total_seconds()
/ 60
        print(
            f"{name}：峰值 {series.max():.3f} bpm，时刻
{series.idxmax():%H:%M}，时刻差 {delay:.0f} 分钟；标准差
{series.std():.3f} bpm"
        )
    print("全局峰值时刻差受峰值削弱和峰形改变影响，不能当作固定相位延
迟。")
    plot_rolling(day, rolling, Config.FIGURES)

if __name__ == "__main__":
    run_step_six(load_heart_rate().loc[Config.DAY])
```